

**Optimización de la gestión de memoria en simuladores cuánticos mediante la
compresión de datos sin pérdida de precisión**

Daniel Adrián González Buendía

Javier Andres Sarmiento Salazar

Trabajo de grado para optar el título de Ingeniero de Sistemas

Director

Luis Alberto Nuñez Martínez

PhD en Ciencias de la Computación

Codirector

Gilberto Javier Díaz Toro

PhD en Ciencias de la Computación

Universidad Industrial de Santander

Facultad De Ingenierías Fisicomecánicas

Escuela De Ingeniería De Sistemas e Informática

Pregrado en Ingeniería de Sistemas e Informática Bucaramanga

2026

Dedicatoria

Este proyecto va dedicado a todas las personas que me apoyaron y aportaron a mi desarrollo como persona, y como profesional. A todas las personas que quiero.

A mis padres.

A mi hermana.

A mis amigos.

A Daniel.

Contenido

Introducción	3
1. Planteamiento del problema	4
2. Objetivos	6
2.1. Objetivo General	6
2.2. Objetivos Específicos	6
3. Marco Teórico	7
3.1. Qubit y Representación del Estado Cuántico	7
3.2. Superposición Cuántica	8
3.3. Entrelazamiento Cuántico	9
3.4. Operadores Unitarios y Evolución del Estado Cuántico	10
3.4.1. Puertas cuánticas como operadores unitarios.	11
3.4.2. Transformaciones en sistemas de múltiples qubits.	12
3.5. Circuitos Cuánticos: Profundidad, Reversibilidad e Interferencia	12
3.5.1. Estructura de un circuito cuántico.	13
3.5.2. Profundidad de un circuito cuántico.	13
3.5.3. Reversibilidad de los circuitos cuánticos.	13
3.5.4. Interferencia cuántica.	14
3.6. Simulación tipo Schrödinger	14
3.7. Partición de Circuitos y Caminos de Feynman	15
3.8. Diagramas de Decisión Cuánticos	15
3.9. Fidelidad entre Estados Cuánticos	16
3.10. Bibliotecas de Compresión con Pérdida: SZ y ZFP	17
4. Estado del Arte	18

4.1. Simuladores Cuánticos	18
4.2. Compresión de Memoria con Pérdida de Precisión	19
4.3. Compresión de Memoria sin Pérdida de Precisión	20
4.4. Algoritmos de compresión sin pérdida de precisión	22
4.4.1. FPC (Burtscher & Ratanaworabhan)	22
4.4.2. FPZIP (Lindstrom & Isenburg)	23
4.4.3. Filtro Bitshuffle con LZ4 (Masui et al.)	24
4.4.4. Transformación de Datos Tipados (TDT)	25
4.5. Otros Métodos de Optimización de Memoria	26
5. Metodología de la Investigación	27
5.1. Diseño	28
5.2. Implementación y optimización del modelo	29
5.3. Evaluación experimental	29
6. Simuladores cuánticos	30
6.1. Motivación y necesidad de un simulador base	30
6.2. Criterios de selección	31
6.3. Alternativas de simuladores evaluadas	33
6.4. Matriz de evaluación ponderada	34
6.5. Selección del simulador: TMFQSfullstate	35
7. Evaluación de algoritmos de compresión sin pérdida para TMFQSfullstate	36
7.1. Motivación y contexto de evaluación	36
7.2. Criterios de selección	36
7.3. Alternativas de compresión sin pérdida consideradas	38
7.4. Matriz de evaluación ponderada	39
7.5. Selección del algoritmo recomendado	40

8. Patrones en vectores de estado de algoritmos cuánticos relevantes para compresión	40
8.1. Introducción	40
8.2. Búsqueda de Grover: amplitudes uniformes y estados de baja entropía	41
8.3. Transformada Cuántica de Fourier: patrones de fase y baja redundancia . . .	41
8.4. Algoritmo de Shor: periodicidad, dispersión y picos	42
8.5. Deutsch–Jozsa y Simon: superposiciones estructuradas y esparsidad	43
8.6. Circuitos variacionales y circuitos aleatorios: entre regularidad y alta entropía	43
8.7. Implicaciones para compresión sin pérdida en simulación de estado completo	44
Referencias	45

Lista de figuras

1. Representación de un estado cuántico sobre la esfera de Bloch. Los polos corresponden a los estados $|0\rangle$ y $|1\rangle$, y los puntos intermedios representan superposiciones. 9
2. Metodología de investigación 28

Lista de tablas

1.	Criterios usados para la selección del simulador cuántico	32
2.	Evaluación de alternativas (escala 1–5) y total ponderado	34
3.	Criterios para la selección de compresores sin pérdida en TMFQSfullstate . .	37
4.	Evaluación de alternativas de compresión (escala 1–5) y total ponderado . .	39

Lista de apéndices

Glosario

IoT: Internet of Things

LDA: Lenguaje de Descripción de Arquitectura (Architecture Description Language)

Resumen

Título: Optimización de la gestión de memoria en simuladores cuánticos mediante la compresión de datos sin pérdida de precisión¹

Autores: Daniel Adrián González Buendía
Javier Andres Sarmiento Salazar²

Palabras clave: Computación cuántica, Simulación cuántica, Gestión de memoria, Compresión de datos, Computación de Alto Rendimiento (HPC)

Descripción:

La simulación de sistemas cuánticos en hardware clásico enfrenta una limitación severa debido al crecimiento exponencial en la memoria requerida. A medida que aumenta el número de qubits, la cantidad de datos a almacenar supera rápidamente las capacidades de las computadoras convencionales y supercomputadoras. Esta restricción impide la exploración de algoritmos cuánticos avanzados y dificulta la validación de experimentos en entornos clásicos antes de su ejecución en hardware cuántico. Por ello, es fundamental desarrollar estrategias eficientes de gestión de memoria que permitan extender el alcance de estas simulaciones sin comprometer la fidelidad de los cálculos. A pesar de los avances en simuladores cuánticos y técnicas de optimización, la gestión de memoria sigue siendo un cuello de botella significativo. En particular, muchas soluciones actuales sacrifican fidelidad a cambio de eficiencia, lo que limita su aplicabilidad en contextos donde la precisión es fundamental. Por ello, es necesario explorar estrategias que reduzcan el consumo de memoria sin comprometer la exactitud de los resultados. Este trabajo propone el diseño e implementación de una estrategia de compresión de memoria sin pérdida de precisión en simuladores cuánticos, con el objetivo de encontrar un balance óptimo entre uso de memoria, rendimiento y fidelidad. Al final, se espera evaluar la viabilidad de la propuesta y compararla con otros enfoques existentes para determinar su impacto en la optimización de simulaciones cuánticas en hardware clásico.

¹Trabajo de Investigación

²Facultad De Ingenierías Fisicomecánicas. Escuela De Ingeniería De Sistemas e Informática.
Director: PhD. Luis Alberto Nuñez Martínez, Codirector: PhD. Gilberto Javier Díaz Toro

Abstract

Title: Optimization of memory management in quantum simulators through data compression without loss of precision³

Authors: Daniel Adrián González Buendía
Javier Andres Sarmiento Salazar⁴

Key words: Quantum Computing, Quantum Simulation, Memory Management, Data Compression, High-Performance Computing (HPC)

Description:

The simulation of quantum systems on classical hardware faces a severe limitation due to the exponential growth in memory requirements. As the number of qubits increases, the amount of data to be stored quickly exceeds the capabilities of conventional computers and supercomputers. This restriction hinders the exploration of advanced quantum algorithms and complicates the validation of experiments in classical environments before their execution on quantum hardware. Therefore, it is crucial to develop efficient memory management strategies that extend the scope of these simulations without compromising computational fidelity. Despite advances in quantum simulators and optimization techniques, memory management remains a significant bottleneck. In particular, many current solutions sacrifice fidelity in exchange for efficiency, limiting their applicability in contexts where precision is crucial. Therefore, it is necessary to explore strategies that reduce memory consumption without compromising the accuracy of the results. This work proposes the design and implementation of a lossless memory compression strategy for quantum simulators, aiming to find an optimal balance between memory usage, performance, and fidelity. Ultimately, the feasibility of the proposal will be evaluated and compared with existing approaches to determine its impact on optimizing quantum simulations on classical hardware.

³Research Work

⁴Faculty of Physics-Mechanics Engineering. School of Systems Engineering and Computer Science.
Advisor: PhD. Luis Alberto Nuñez Martínez, Co-Advisor: PhD. Gilberto Javier Díaz Toro

Introducción

La simulación cuántica en computadoras clásicas enfrenta un desafío fundamental debido al crecimiento exponencial del espacio de estados conforme aumenta el número de qubits. Por ejemplo, la representación de un sistema cuántico de 40 qubits requiere aproximadamente $8 \text{ bytes (double)} \times 2 \text{ (real, imag)} \times 2^{40} = 16 \text{ TB}$ de memoria RAM, mientras que para 50 qubits la demanda se dispara a petabytes (PB), superando la capacidad de las computadoras convencionales (Wu y cols., 2019a). Esta limitación de memoria restringe el estudio de algoritmos cuánticos avanzados y su validación en hardware clásico antes de su implementación en procesadores cuánticos (Zhang y cols., 2024a). Ante esta problemática, es necesario desarrollar estrategias eficientes de gestión de memoria que permitan extender el alcance de las simulaciones cuánticas sin comprometer la viabilidad computacional.

Considerando la creciente importancia de la computación cuántica, optimizar la memoria en simuladores cuánticos no solo permite ampliar el rango de algoritmos simulables en hardware clásico, sino que también reduce la dependencia exclusiva de procesadores cuánticos experimentales. Además, una gestión eficiente de memoria podría reducir significativamente los costos computacionales y el tiempo de simulación en supercomputadoras, lo que resulta en un beneficio directo tanto para la investigación en computación cuántica como para aplicaciones prácticas en la industria y la ciencia.

Para abordar este problema, se han explorado diversas estrategias que pueden agruparse en varias familias de métodos. Entre ellas, la compresión de datos ha sido ampliamente estudiada con enfoques como la compresión con pérdida controlada (e.g., SZ, ZFP), que permite reducir la carga de almacenamiento de vectores de estado sin afectar significativamente la fidelidad de los cálculos (Wu, Di, Cappello, y cols., 2018). Otra línea de investigación relevante es la de los métodos de podado de estados cuánticos, los cuales eliminan amplitudes irrelevantes para disminuir el consumo de memoria, aunque con un impacto variable en la precisión del resultado (Song, Li, y Wu, 2023). Asimismo, la computación de alto rendimien-

to (HPC o High Performance Computing) ha demostrado ser una solución clave, empleando distribución de datos entre nodos y optimizaciones en el acceso a memoria para maximizar la eficiencia de la simulación en infraestructuras de supercomputadoras (Díaz, Steffenel, Barrios, y Couturier, 2024).

A pesar de los avances en estas áreas, muchas de estas estrategias introducen errores acumulativos en la simulación, lo que puede comprometer la validez de los resultados. En este contexto, esta investigación propone una estrategia basada en compresión de datos sin pérdida de precisión, con el objetivo de optimizar el almacenamiento de vectores de estado sin alterar las amplitudes cuánticas originales. Para ello, el proyecto se desarrollará en varias etapas. Inicialmente, se llevará a cabo una revisión del estado del arte para identificar las técnicas de compresión más relevantes. Luego, se seleccionará un simulador cuántico adecuado que permita modificaciones en su gestión de memoria para evaluar dichas técnicas. Posteriormente, se diseñará una estrategia para seleccionar e integrar técnicas de compresión sin pérdida de precisión existentes, seguida de su implementación y optimización en el simulador elegido. Finalmente, se realizarán experimentos controlados para evaluar el impacto de la compresión en términos de ahorro de memoria, tiempo de simulación y fidelidad del resultado, comparándolo con enfoques tradicionales. En las siguientes secciones, se detallarán cada una de estas etapas y su contribución al objetivo final del proyecto.

1 Planteamiento del problema

La simulación de sistemas cuánticos en computadoras clásicas se enfrenta a una barrera casi insuperable: el crecimiento exponencial de la memoria requerida. Representar un estado cuántico de n qubits implica almacenar 2^n números complejos, lo que rápidamente supera la capacidad de cualquier sistema de memoria convencional. Las técnicas actuales, como la poda de estados, reducen drásticamente el consumo de recursos pero a costa de la precisión, mientras que la compresión con pérdida, aunque ofrece mayores índices de reducción, sacrifica irremediabilmente la fidelidad de los cálculos. Este dilema —la imposibilidad de simular

algoritmos cuánticos complejos sin perder información crítica— constituye el núcleo del problema, amenazando la validez y aplicabilidad de las simulaciones en el desarrollo de nuevas estrategias cuánticas.

Este desafío se agrava al considerar que la simulación precisa es esencial para la validación y desarrollo de algoritmos cuánticos en entornos clásicos. La degradación de la precisión debido a la pérdida de datos no solo impide obtener resultados confiables, sino que también limita la escalabilidad y el rendimiento de los simuladores cuánticos. Por ello, es crucial explorar alternativas que permitan optimizar el uso de recursos computacionales sin comprometer la exactitud de los cálculos, destacando la urgente necesidad de investigar métodos de compresión sin pérdida que ofrezcan un equilibrio entre eficiencia y fidelidad en la simulación cuántica.

Con este planteamiento se proponen los siguientes objetivos para abordar esta problemática.

2 Objetivos

2.1 Objetivo General

- Diseñar e implementar una estrategia eficiente de gestión de memoria en simuladores cuánticos basada en técnicas de compresión sin pérdida de precisión, con el fin de optimizar el uso de recursos computacionales sin comprometer la fidelidad de las simulaciones.

2.2 Objetivos Específicos

- Analizar el impacto del uso de herramientas de compresión de datos sin pérdida de precisión en el consumo de memoria y precisión de los resultados en simuladores cuánticos.
- Diseñar una estrategia integral de gestión de memoria basada en compresión sin pérdida de precisión, definiendo el algoritmo, los parámetros de compresión-descompresión y los puntos de integración con el simulador cuántico seleccionado.
- Desarrollar e integrar la estrategia diseñada en el simulador elegido, optimizando la sobrecarga computacional para garantizar un ahorro significativo de memoria sin afectar la fidelidad de la simulación.
- Implementar dos algoritmos cuánticos distintos en un simulador cuántico para evaluar el uso de memoria, velocidad y precisión con y sin compresión de datos.
- Comparar los resultados obtenidos con estrategias tradicionales y métodos que emplean compresión con pérdida de precisión, identificando ventajas y desventajas de cada enfoque.

3 Marco Teórico

La computación cuántica representa un nuevo paradigma en el procesamiento de información, basado en fenómenos propios de la mecánica cuántica como la superposición, el entrelazamiento y la interferencia. Comprender estos conceptos es esencial para abordar los retos que plantea su simulación en computadoras clásicas y para justificar el uso de técnicas avanzadas como la compresión de datos sin pérdida. A continuación, se presentan los fundamentos teóricos necesarios para contextualizar el problema abordado en este trabajo.

3.1 Qubit y Representación del Estado Cuántico

Un *qubit* o *bit cuántico* es la unidad básica de información en computación cuántica. A diferencia del bit clásico, que solo puede representar los valores 0 o 1, un qubit puede encontrarse en una superposición lineal de ambos estados. Matemáticamente, su estado se expresa como:

$$|\psi\rangle = \alpha_1 |0\rangle + \alpha_2 |1\rangle$$

donde $\alpha_1, \alpha_2 \in \mathbb{C}$ y $|\alpha_1|^2 + |\alpha_2|^2 = 1$. Esta condición garantiza que la probabilidad de encontrar al qubit en el estado $|0\rangle$ o en el estado $|1\rangle$ sea del 100%. Cuando se tienen n qubits, el sistema completo se representa como un vector de 2^n amplitudes complejas en un espacio de Hilbert de dimensión 2^n . Para $n = 3$ hay $2^3 = 8$ estados computacionales. El estado general de 3 qubits puede escribirse como:

$$|\psi\rangle = \sum_{i=0}^7 \alpha_i |i\rangle, \quad \text{con } |i\rangle \in \{|000\rangle, |001\rangle, \dots, |111\rangle\}$$

Aquí, $\alpha_i \in \mathbb{C}$ es la amplitud asociada al estado base $|i\rangle$ y satisface la condición de normalización $\sum_{i=0}^{2^n-1} |\alpha_i|^2 = 1$. Cada amplitud compleja en doble precisión ocupa 16 bytes (8 bytes para la parte real y 8 bytes para la imaginaria), lo que impone una gran carga de memoria cuando n crece.

3.2 Superposición Cuántica

La *superposición* es una de las propiedades fundamentales de la mecánica cuántica. Consiste en la capacidad de un sistema cuántico de existir simultáneamente en múltiples estados posibles antes de ser observado o medido. En el caso de un qubit, esta propiedad se expresa formalmente como:

$$|\psi\rangle = \alpha_1 |0\rangle + \alpha_2 |1\rangle, \quad \text{con } \alpha_1, \alpha_2 \in \mathbb{C} \text{ y } |\alpha_1|^2 + |\alpha_2|^2 = 1$$

Aquí, $|0\rangle$ y $|1\rangle$ representan los estados clásicos posibles del qubit, mientras que α_1 y α_2 son las amplitudes de probabilidad asociadas a cada estado. Aunque el sistema ‘está’ en ambos estados al mismo tiempo, una vez se realiza una medición, colapsa a uno solo de ellos con probabilidad $|\alpha_1|^2$ o $|\alpha_2|^2$, respectivamente. Desde una perspectiva geométrica, el estado de un qubit puede representarse como un punto en la esfera de Bloch, donde cada combinación válida de α_1 y α_2 corresponde a una ubicación en la superficie de la esfera. Esta representación visual resulta útil para entender cómo las puertas cuánticas rotan el estado en dicho espacio.

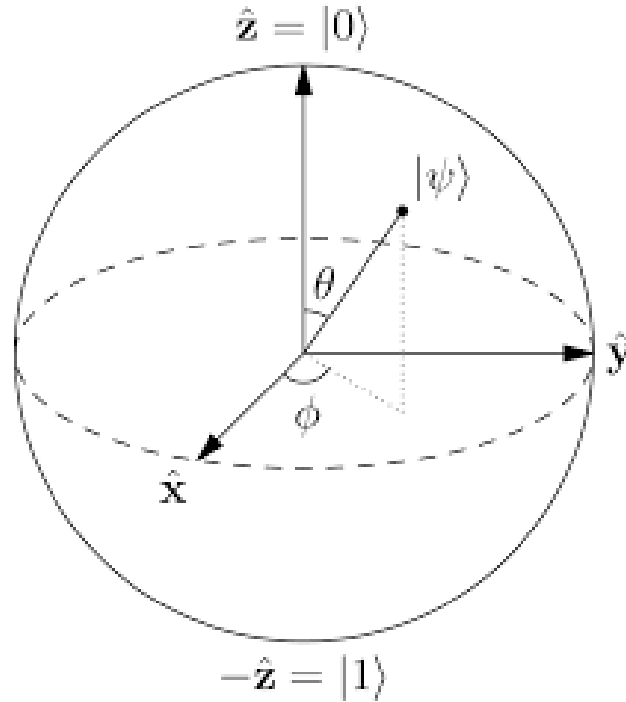


Figura 1: Representación de un estado cuántico sobre la esfera de Bloch. Los polos corresponden a los estados $|0\rangle$ y $|1\rangle$, y los puntos intermedios representan superposiciones.

En términos computacionales, la superposición permite que un sistema de n qubits represente, de manera simultánea, todos los 2^n posibles estados clásicos, lo cual es el fundamento del *paralelismo cuántico*. Este fenómeno, aunque no equivale directamente a realizar 2^n cálculos a la vez, permite evaluar de forma concurrente múltiples configuraciones de entrada, y es lo que hace potencialmente tan poderosos a los algoritmos cuánticos.

3.3 Entrelazamiento Cuántico

El *entrelazamiento* cuántico es un fenómeno mediante el cual dos o más qubits quedan conectados de tal manera que el estado de uno de ellos no puede describirse independientemente del estado del otro, sin importar cuán lejos se encuentren en el espacio. Esta correlación es más fuerte que cualquier correlación clásica y desafía las intuiciones tradicionales sobre causalidad y localidad. Formalmente, un sistema de dos qubits está entrelazado si su estado no puede expresarse como el producto tensorial de dos estados individuales. Por ejemplo, el

siguiente estado de Bell:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

En esta expresión, el factor $1/\sqrt{2}$ garantiza la normalización del estado, ya que $\left|\frac{1}{\sqrt{2}}\right|^2 + \left|\frac{1}{\sqrt{2}}\right|^2 = 1$. El signo “+” indica una superposición lineal (suma vectorial en el espacio de Hilbert) de los estados base $|00\rangle$ y $|11\rangle$; no corresponde a concatenación ni a una operación lógica clásica.

Este estado de Bell no puede escribirse como $|\psi_1\rangle \otimes |\psi_2\rangle$. Esto significa que no existe una descripción completa del sistema que consista simplemente en describir el estado de cada qubit por separado. Este tipo de correlación permite que la medición de uno de los qubits determine instantáneamente el estado del otro, incluso si están separados por distancias arbitrariamente grandes. Sin embargo, este efecto no viola la relatividad ni permite transmitir información más rápido que la luz, ya que el resultado de cada medición individual es aleatorio. El entrelazamiento es un recurso clave en muchos algoritmos cuánticos avanzados, en protocolos de teleportación cuántica, y en esquemas de criptografía cuántica como el intercambio de claves cuánticas (Quantum Key Distribution, QKD). Además, es una de las principales fuentes de complejidad en la simulación de sistemas cuánticos en hardware clásico, pues aumenta de forma significativa el entrecruzamiento de información entre los qubits.

3.4 Operadores Unitarios y Evolución del Estado Cuántico

La evolución de un sistema cuántico aislado se describe mediante la mecánica cuántica a través de la ecuación de Schrödinger. Esta evolución es determinista, reversible y está gobernada por operadores matemáticos denominados *operadores unitarios*. En términos generales, si el estado inicial de un sistema es $|\psi(0)\rangle$, su evolución en el tiempo t viene dada por la aplicación de un operador unitario $U(t)$:

$$|\psi(t)\rangle = U(t) |\psi(0)\rangle$$

Donde $U(t)$ es una matriz unitaria que cumple:

$$U^\dagger U = U U^\dagger = I$$

Aquí, $U^\dagger$ representa el conjugado transpuesto (o adjunto) de U , e I es la matriz identidad. Esta condición garantiza que la norma del estado cuántico se conserva durante la evolución, lo que implica que la probabilidad total sigue siendo 1. Además, asegura la *reversibilidad* de las operaciones, una característica fundamental de la dinámica cuántica (con excepción del proceso de medición).

3.4.1 Puertas cuánticas como operadores unitarios.

En computación cuántica, las operaciones que transforman los estados de los qubits se conocen como *puertas cuánticas*. Estas puertas son implementaciones físicas y algorítmicas de operadores unitarios y constituyen los bloques fundamentales para construir circuitos cuánticos. Cada puerta cuántica puede representarse mediante una matriz unitaria. Por ejemplo, la puerta Hadamard (H), que genera una superposición equitativa, se define como:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Aplicar H al estado base $|0\rangle$ produce:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Este resultado es una superposición con amplitudes iguales de los estados $|0\rangle$ y $|1\rangle$, un recurso clave en algoritmos como el de Grover o Deutsch-Jozsa.

3.4.2 Transformaciones en sistemas de múltiples qubits.

En sistemas con varios qubits, los operadores unitarios pueden actuar sobre uno o más qubits. Por ejemplo, la puerta CNOT (control-NOT) es una operación de dos qubits que invierte el segundo qubit (qubit objetivo) si el primero (qubit de control) está en el estado $|1\rangle$. Esta operación es esencial para generar entrelazamiento entre qubits. Para aplicar una puerta a un sistema completo de n qubits, se utiliza el producto tensorial de matrices. Por ejemplo, para aplicar una puerta G al tercer qubit en un sistema de 5 qubits, se construye:

$$U = I \otimes I \otimes G \otimes I \otimes I$$

La dimensión total de la matriz U será $2^5 \times 2^5 = 32 \times 32$, lo que ilustra el rápido crecimiento de la complejidad computacional al aumentar el número de qubits.

3.5 Circuitos Cuánticos: Profundidad, Reversibilidad e Interferencia

La computación cuántica, en su forma más común, se modela a través del llamado *modelo de circuitos cuánticos*. En este enfoque, un algoritmo cuántico se representa como una secuencia de operaciones —puertas cuánticas— aplicadas a un conjunto inicial de qubits. El circuito cuántico se construye de manera análoga a los circuitos digitales clásicos, pero con componentes que manipulan estados cuánticos en lugar de bits clásicos.

3.5.1 Estructura de un circuito cuántico.

Formalmente, un circuito cuántico está compuesto por tres fases:

1. **Inicialización:** Se prepara el sistema en un estado base, usualmente $|0\rangle^{\otimes n}$.
2. **Aplicación de puertas cuánticas:** Se aplica una secuencia de operaciones unitarias sobre los qubits.
3. **Medición:** Se colapsa el sistema a un resultado clásico mediante la observación.

Cada puerta cuántica actúa sobre uno o varios qubits, y las operaciones se disponen en capas, donde una capa puede contener varias puertas que se aplican en paralelo a qubits distintos.

3.5.2 Profundidad de un circuito cuántico.

La *profundidad* de un circuito se refiere al número de capas secuenciales de operaciones que deben ejecutarse en orden, considerando que algunas pueden realizarse en paralelo. Un circuito de menor profundidad implica que el algoritmo se ejecuta en menos pasos cuánticos, lo cual es deseable por dos razones principales:

- Reduce la duración total del algoritmo, minimizando la exposición del sistema a errores por decoherencia.
- Mejora la viabilidad de implementación en dispositivos cuánticos actuales, que están limitados en fidelidad y tiempo de coherencia.

Por ejemplo, un circuito que aplica n puertas de una sola vez en paralelo tiene una profundidad de 1, mientras que si cada puerta debe aplicarse una tras otra, la profundidad será n . Así, la profundidad es una medida directa de la complejidad temporal del algoritmo.

3.5.3 Reversibilidad de los circuitos cuánticos.

Toda operación cuántica (exceptuando la medición) debe ser *reversible*, lo que significa que debe existir una operación inversa que recupere el estado anterior sin pérdida de in-

formación. Esto se garantiza matemáticamente porque las puertas cuánticas se representan mediante operadores unitarios, los cuales por definición son invertibles. Esta propiedad distingue a los circuitos cuánticos de los circuitos clásicos, donde muchas operaciones (como AND u OR) son irreversibles. En el mundo cuántico, la única operación no reversible es la medición, que colapsa el estado superpuesto a uno de sus posibles resultados y destruye la información contenida en las amplitudes de probabilidad. La reversibilidad también es esencial para ciertos algoritmos cuánticos que requieren ‘deshacer’ operaciones intermedias sin borrar información, como ocurre en la técnica de *descomputación* (uncomputation), utilizada para limpiar registros auxiliares antes de la medición.

3.5.4 Interferencia cuántica.

La *interferencia* cuántica es un fenómeno exclusivo de los sistemas que evolucionan bajo la mecánica cuántica. En este contexto, las amplitudes de probabilidad asociadas a distintos caminos de evolución del sistema pueden combinarse de forma constructiva (reforzando la probabilidad de ciertos resultados) o destructiva (cancelando otros resultados). Esta propiedad es esencial para el funcionamiento de muchos algoritmos cuánticos. Por ejemplo, en el algoritmo de Grover para búsqueda no estructurada, las soluciones correctas se refuerzan mediante interferencia constructiva, mientras que las opciones incorrectas se atenúan mediante interferencia destructiva. A diferencia de la superposición, que permite explorar múltiples estados al mismo tiempo, la interferencia actúa como un mecanismo de filtrado que permite extraer los resultados deseados al final del proceso computacional. Es la combinación de ambas —superposición e interferencia— lo que da lugar a la aceleración cuántica.

3.6 Simulación tipo Schrödinger

La simulación tipo Schrödinger es una de las aproximaciones más utilizadas en la computación cuántica para modelar la evolución de un sistema de qubits. Este método consiste en almacenar el vector de estado completo del sistema cuántico, que contiene 2^n amplitudes

complejas para un sistema de n qubits. En cada paso del circuito, se aplican directamente las puertas cuánticas como transformaciones matriciales sobre este vector. Este enfoque ofrece una representación exacta del sistema y permite la simulación precisa de cualquier circuito cuántico, independientemente del grado de entrelazamiento o la profundidad del circuito. Sin embargo, su principal limitación es el consumo de memoria, que crece exponencialmente con el número de qubits lo cual es impracticable para la mayoría de los sistemas.

3.7 Partición de Circuitos y Caminos de Feynman

Una alternativa a la simulación tipo Schrödinger es el uso de técnicas basadas en *partición de circuitos* y *caminos de Feynman*. Estos métodos dividen el circuito cuántico en subcircuitos más pequeños o regiones menos entrelazadas, reduciendo así los requerimientos de memoria al costo de mayor tiempo de cómputo. La técnica de caminos de Feynman interpreta la evolución del sistema como una suma sobre todos los caminos computacionales posibles que conectan los estados iniciales y finales. Esta formulación, si bien elegante y más eficiente en términos de espacio, conlleva un crecimiento exponencial del número de caminos a evaluar, especialmente cuando la profundidad del circuito es alta. Estos enfoques han sido utilizados exitosamente para simular circuitos de hasta 54 qubits, principalmente adoptando soluciones híbridas con las simulaciones de Schrödinger, obteniendo de esta manera una reducción significativa en el uso de la memoria, sin sacrificar rendimiento (Arute y cols., 2019; Boixo, Isakov, Smelyanskiy, y Neven, 2017; Markov, Fatima, Isakov, y Boixo, 2018; Pednault y cols., 2020).

3.8 Diagramas de Decisión Cuánticos

Los *diagramas de decisión cuánticos* son estructuras de datos jerárquicas inspiradas en los diagramas de decisión binarios (BDDs) utilizados en verificación formal y diseño lógico. Estas estructuras permiten representar de forma comprimida vectores de estado y matrices unitarias aprovechando redundancias y patrones repetitivos.

- **QuIDD (Quantum Information Decision Diagrams)** codifican vectores y operadores mediante nodos reutilizables, lo que permite representar estructuras altamente simétricas de manera eficiente.
- **QMDD (Quantum Multiple-valued Decision Diagrams)** extienden los QuIDD utilizando aristas ponderadas con matrices, lo que los hace más aptos para representar operadores complejos y circuitos reversibles.
- **LIMDD (Local Invertible Map Decision Diagrams)** introducen transformaciones locales invertibles como aristas, lo cual permite representar incluso estados estabilizadores y circuitos con entrelazamiento moderado de manera eficiente.

Estas estructuras son altamente efectivas cuando el circuito presenta redundancia estructural, permitiendo pasar de una representación exponencial a una polinomial. Sin embargo, pierden eficacia en circuitos con alta interferencia o sin regularidad estructural, como ocurre en simulaciones aleatorias o algoritmos de propósito general.

3.9 Fidelidad entre Estados Cuánticos

Una métrica fundamental para evaluar la similitud entre dos estados cuánticos es la *fidelidad*. Generalmente, se emplea para cuantificar la diferencia entre dos estados cuánticos. Dado dos estados puros $|\psi_r\rangle$ (referencia) y $|\psi_c\rangle$ (estado simulado), la fidelidad entre ellos se define como:

$$F(|\psi_r\rangle, |\psi_c\rangle) = |\langle\psi_r|\psi_c\rangle|^2.$$

Esta expresión mide el cuadrado del valor absoluto del producto interno entre ambos vectores. La fidelidad toma valores entre 0 y 1, donde:

- $F = 1$ indica que los estados son idénticos.
- $F = 0$ indica que son ortogonales (completamente diferentes).
- Valores intermedios indican distintos grados de similitud.

Por ejemplo, si:

$$|\psi_r\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \quad |\psi_c\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 0,99 \\ 0 \\ 0 \\ 1,01 \end{bmatrix}$$

entonces, normalizando y aplicando la fórmula, se obtiene una fidelidad cercana a 1, lo que indica que el estado simulado es prácticamente indistinguible del original, a pesar de las pequeñas diferencias numéricas. Esta métrica es especialmente útil para evaluar estrategias de compresión: una fidelidad igual a 1 garantiza que el proceso de compresión y descompresión no ha alterado el estado cuántico original.

3.10 Bibliotecas de Compresión con Pérdida: SZ y ZFP

Las bibliotecas **SZ** y **ZFP** son herramientas de compresión científica ampliamente utilizadas para reducir el tamaño de datos en punto flotante sin necesidad de conservar todos los dígitos exactos.

- **SZ** aplica una estrategia de predicción adaptativa con control de error. Estima el valor de cada dato basado en sus vecinos, almacena solo la diferencia si esta cae dentro de un margen de error aceptable, y utiliza codificación entropica para comprimir los residuos. Es muy eficiente cuando los datos presentan correlaciones locales.
- **ZFP** divide los datos en bloques pequeños y los transforma a un dominio más compresible mediante una técnica similar a la transformada discreta del coseno. Posteriormente, se aplican técnicas de truncamiento y codificación por bloques. ZFP permite controlar la compresión por tasa fija, error absoluto o fidelidad relativa.

4 Estado del Arte

4.1 Simuladores Cuánticos

La simulación clásica de circuitos cuánticos es esencial para el desarrollo y validación de algoritmos cuánticos. Sin embargo, el principal desafío radica en la representación del estado cuántico, cuyo tamaño crece exponencialmente con el número de qubits.

Uno de los primeros enfoques para abordar este problema fue la representación mediante *Matrix Product States* (MPS), propuesta por Vidal (2003). Este método permite representar ciertos estados cuánticos de forma eficiente cuando el entrelazamiento es limitado, reduciendo los requisitos de almacenamiento y cómputo. Su principal ventaja es la escalabilidad en circuitos con baja profundidad, permitiendo simular cientos de qubits. No obstante, su limitación radica en que para sistemas altamente entrelazados el costo computacional crece exponencialmente, restringiendo su aplicabilidad a ciertos algoritmos.

A medida que se buscaban métodos más generales, surgió la simulación tipo *Schrödinger*, que almacena el vector de estado completo y lo actualiza en cada operación cuántica. Este enfoque, empleado en simuladores como QuEST y Qiskit Aer (Li, Kimura, Sato, Yu, y Fujita, 2023), garantiza alta precisión y permite simular cualquier circuito cuántico. Su desventaja principal es la extrema demanda de memoria, la cual crece exponencialmente con el número de qubits, alcanzando 0.5 petabytes para 45 qubits y varios exabytes para 50 qubits (Pednault y cols., 2020).

Para mitigar el problema de memoria, se desarrollaron métodos basados en partición del circuito y caminos de Feynman. Por ejemplo, Google e IBM utilizaron esta técnica para simular circuitos de hasta 56 qubits con profundidad limitada (Chen, Zhang, Huang, Newman, y Shi, 2018). Sin embargo, aunque reducen el almacenamiento requerido, estos métodos incrementan la complejidad computacional, ya que el número de caminos crece exponencialmente con la profundidad del circuito.

4.2 Compresión de Memoria con Pérdida de Precisión

Dado que la representación exacta de estados cuánticos es ineficiente en términos de memoria, se han explorado técnicas de compresión con pérdida de precisión. Estos enfoques sacrifican una pequeña cantidad de fidelidad numérica a cambio de una reducción significativa en el uso de memoria, permitiendo la simulación de circuitos más grandes.

Uno de los primeros enfoques en este ámbito fue el de Wu, Di, Cappello, y cols. (2018); Wu y cols. (2019a), quienes introdujeron la compresión adaptativa con error acotado. Utilizando la biblioteca SZ, lograron reducir el tamaño del vector de estado hasta en un factor de 16, manteniendo fidelidades superiores al 99.9 %. Su principal ventaja es que permite ampliar el número de qubits simulados sin un incremento proporcional en los requisitos de memoria. Sin embargo, la compresión y descompresión recurrente introduce sobrecarga computacional, afectando la eficiencia temporal de la simulación.

En un esfuerzo por maximizar la escala de la simulación cuántica de estado completo, Zhang y cols. (2024a) presentaron *BMQSim*, un marco que extiende *SV-Sim* mediante el uso de **compresores con pérdida de precisión y control de error punto a punto** ejecutados directamente en GPU (por ejemplo, variantes de la familia SZ/cuSZ)(Zhang y cols., 2024b). Su diseño combina:

- Una estrategia de *circuit partitioning* que disminuye la frecuencia de (des)compresión.
- Un *pipeline* solapado que oculta en gran medida el coste de mover y comprimir datos.
- Una gestión de memoria jerárquica (VRAM + SSD mediante GPUDirect Storage) que asigna dinámicamente espacio a los bloques comprimidos.

Con estas técnicas, *BMQSim* reduce el consumo de memoria en más de un orden de magnitud y mantiene fidelidades superiores al 99 % sin penalizar de forma apreciable el tiempo de simulación. Su eficacia, sin embargo, está condicionada a la disponibilidad de hardware acelerado con GPUs de alto rendimiento y unidades NVMe rápidas, lo que puede restringir su adopción en plataformas de menor capacidad computacional.

Más recientemente, Huffman, Pavlichin, y Weissman (2024) exploraron la cuantización de amplitudes como un mecanismo para reducir la precisión numérica sin afectar la fidelidad global del sistema. Demostraron que representaciones con tan solo 7 bits por amplitud permiten mantener una fidelidad superior al 99 %, lo que sugiere que la precisión completa de punto flotante puede ser innecesaria en muchas simulaciones. Aunque esta técnica ofrece un enfoque simple y eficiente para reducir el uso de memoria, su aplicabilidad a circuitos de gran escala aún requiere validación empírica.

Díaz Toro (2025) propone un esquema de **vector de estado comprimido** que emplea compresores con pérdida de precisión—principalmente *ZFP* en modo *fixed-rate*, complementado por *SZ*—para reducir la huella de memoria del simulador. El autor reporta reducciones del **10–20 %** en casos base y, en configuraciones de mayor escala, ahorros que alcanzan hasta un **75 %** de la memoria requerida. *ZFP* opera sobre bloques independientes de 4^d valores, de modo que la (des)compresión de cada bloque se realiza en completo aislamiento del resto; esto permite solapar dichas operaciones con el cómputo principal y amortiguar la sobrecarga introducida. Aunque la compresión incrementa el tiempo de ejecución, el impacto se compensa al transmitir los fragmentos del vector en formato comprimido dentro de una arquitectura distribuida basada en *MPI*, reduciendo el volumen de comunicación y habilitando simulaciones de hasta 30 qubits en los experimentos reportados. Finalmente, el estudio concluye que esta estrategia de *estado completo comprimido distribuido* resulta más fiable y escalable que la *poda de estados de baja probabilidad*, la cual presenta descensos de fidelidad y sobrecarga adicional que la hacen desaconsejable.

4.3 Compresión de Memoria sin Pérdida de Precisión

La simulación cuántica eficiente requiere reducir el consumo de memoria sin comprometer la fidelidad del estado cuántico. Para ello, se han desarrollado técnicas de compresión sin pérdida de precisión que buscan representar estados y operadores cuánticos de manera más compacta sin alterar sus valores.

Uno de los primeros enfoques en esta dirección fue el uso de diagramas de decisión cuánticos (*Quantum Information Decision Diagrams*, QuIDD), introducidos por Viamontes, Markov, y Hayes (2005). Este método permite representar vectores de estado y operadores unitarios aprovechando redundancias estructurales en sus coeficientes, lo que permite una representación más eficiente en memoria. Su ventaja radica en que, para circuitos con alta redundancia, el crecimiento en memoria puede reducirse de exponencial a polinomial. Sin embargo, su eficacia se limita a circuitos con estructura repetitiva, fallando en casos de entrelazamiento complejo.

En un desarrollo posterior, Miller y Thornton (2006) propusieron los *Quantum Multiple-valued Decision Diagrams* (QMDD), los cuales extienden la representación de QuIDD mediante el uso de aristas ponderadas con matrices unitarias. Esto permitió manejar de manera más eficiente circuitos reversibles y operadores cuánticos de mayor tamaño. Aunque estos diagramas optimizan la representación de puertas lógicas y matrices densas, su eficiencia sigue dependiendo de la estructura interna del circuito.

Más recientemente, Jaques y Häner (2021) exploraron la esparsidad del estado cuántico para almacenar solo las amplitudes no nulas, reduciendo significativamente el almacenamiento requerido en algoritmos con exploración limitada del espacio de Hilbert. Su enfoque demostró ser altamente eficiente en problemas de factorización y aritmética cuántica, pero pierde efectividad en circuitos con alta interferencia y entrelazamiento.

En 2023, Vinkhuijzen, Coopmans, Elkouss, Dunjko, y Laarman (2023) introdujeron los *Local Invertible Map Decision Diagrams* (LIMDD), una evolución de los diagramas de decisión que incorpora transformaciones locales para mejorar la representación compacta del estado cuántico. Esta técnica permite manejar estados estabilizadores y ciertas estructuras altamente entrelazadas que eran difíciles de representar con diagramas de decisión tradicionales. No obstante, su aplicabilidad sigue limitada a casos donde las transformaciones locales puedan ser explotadas para reducir la complejidad estructural.

Estos avances en compresión sin pérdida han permitido extender la capacidad de simu-

lación de circuitos cuánticos en hardware clásico. Sin embargo, aún existen limitaciones en la representación de estados arbitrarios, lo que ha llevado a la exploración de otros métodos para optimizar el uso de memoria.

4.4 Algoritmos de compresión sin pérdida de precisión

En simulaciones científicas y cuánticas es común manejar vectores de estado representados como grandes arreglos de números de punto flotante. Cuando estos datos carecen de correlación temporal o patrones estructurales (es decir, presentan alta entropía o *aleatoriedad*), la compresión sin pérdida se vuelve especialmente desafiante. Muchos algoritmos de compresión de punto flotante aprovechan transiciones suaves o redundancias entre valores consecutivos; por ejemplo, técnicas de serie temporal almacenan solo los bits que difieren entre un valor y el siguiente, explotando que a menudo los valores sucesivos comparten prefijos de bits iguales. Sin embargo, en datos altamente aleatorios, estas suposiciones se rompen y tales métodos tienden a no lograr compresión significativa. Por lo tanto, es crucial identificar y seleccionar algoritmos de compresión sin pérdida que sean efectivos para datos de punto flotante con estas características. A continuación, se describen los principales algoritmos que han sido concebidos para este tipo de datos, analizando su funcionamiento, rendimiento y pertinencia para ser implementados y evaluados en el contexto de este trabajo de grado, con el fin de encontrar un balance óptimo entre el ahorro de memoria y la eficiencia computacional.

4.4.1 *FPC (Burtscher & Ratanaworabhan)*

El algoritmo *FPC (Floating Point Compressor)* (Burtscher y Ratanaworabhan, 2009) fue diseñado para la compresión sin pérdida de flujos lineales de datos en doble precisión (64 bits). Internamente, FPC emplea un esquema de *predicción* dual junto con codificación de ceros líderes. Concretamente, mantiene dos predictores (derivados de técnicas FCM/DFCM) almacenados en tablas hash, que generan un valor estimado para el siguiente número en la secuencia. Para cada nuevo valor de entrada, FPC selecciona la predicción que más coincide

en sus bits más significativos con el valor real, y calcula el **XOR** entre el valor real y el predicho. Este **XOR** resalta únicamente los bits diferentes, convirtiendo los bits coincidentes en *cero*. A continuación, el algoritmo cuenta cuántos bytes iniciales del resultado **XOR** son cero (ceros líderes) y codifica esa cantidad en 3 bits, junto con 1 bit adicional que indica cuál predictor fue usado. Estos 4 bits de código, seguidos de los *bytes residuales* no nulos (almacenados sin más codificación), forman la salida comprimida.

Esta estrategia implica que, si el valor predicho es cercano al real, la diferencia tendrá muchos ceros líderes y, por tanto, pocos bytes residuales que almacenar. En escenarios de datos altamente aleatorios (donde las predicciones rara vez aciertan), FPC tiende a obtener pocos ceros líderes y la tasa de compresión se aproxima a 1:1 (sin reducción notable). No obstante, su diseño minimiza la penalización en estos casos: los bloques comprimidos incluyen encabezados mínimos y códigos de apenas 4 bits por valor, garantizando sobrecarga reducida y un rendimiento consistente. Así, aunque FPC no logre compresión sustancial en vectores de estado cuántico aleatorios, ofrece una solución de alta velocidad con mínima sobrecarga, capaz de detectar rápidamente si no hay redundancia aprovechable y manejar dichos datos sin degradar el rendimiento.

4.4.2 **FPZIP** (*Lindstrom & Isenburg*)

FPZIP (Lindstrom y Isenburg, 2006) es un compresor de propósito específico para arreglos de punto flotante. Su enfoque se basa en la *predicción adaptativa* y la *codificación entrópica* eficiente para lograr compresión sin pérdida en múltiples contextos (mallas no estructuradas, volúmenes 3D, imágenes, etc.). Internamente, FPZIP aplica esquemas de predicción multi-dimensional (por ejemplo, predictores que aprovechan la continuidad espacial de los datos científicos) y luego comprime únicamente el error de predicción. A diferencia de métodos que cuantizan los flotantes a enteros, FPZIP opera directamente sobre las representaciones IEEE 754, manteniendo exactitud bit a bit.

Su arquitectura soporta predictores ajustables al tipo de datos: por ejemplo, en un campo

volumétrico 3D puede usar un predictor basado en valores vecinos, mientras que en una serie temporal podría usar extrapolación simple. Tras la predicción, FPZIP emplea un modelo de codificación entrópica optimizado para velocidad, que comprime los residuos de manera eficaz. Esta combinación le permite alcanzar tasas de compresión y velocidades competitivas, acelerando la E/S de simulaciones científicas al reducir el volumen de datos sin sacrificar precisión. En contextos de datos altamente aleatorios, donde las predicciones en su mayoría fallan, FPZIP tiende a comportarse como un compresor genérico pero con ventajas sutiles: incluso en ausencia de correlación temporal, puede explotar patrones locales dentro de la representación en coma flotante (p. ej., regularidades en campos de exponentes o signos comunes) que los algoritmos genéricos pasan por alto al tratar los datos solo como bytes crudos. Para vectores de estado cuántico, si bien los amplitudes pueden lucir aleatorios, FPZIP podría aprovechar distribuciones no uniformes de ciertos bits (por ejemplo, exponentes repetidos), logrando compresiones modestas con un rendimiento robusto.

4.4.3 Filtro Bitshuffle con LZ4 (Masui et al.)

Una estrategia distinta para afrontar datos de punto flotante sin patrones evidentes es emplear transformaciones previas que revelen posibles regularidades a nivel de bits. *Bitshuffle* (Masui y cols., 2015) es un algoritmo de preprocesamiento sin pérdida que extiende el filtro *byte-shuffle* al nivel de *bit*. En lugar de reordenar solo los bytes de cada número (como hace el filtro *shuffle* de HDF5), Bitshuffle realiza una *transposición bit a bit* de una matriz de valores: considera la representación binaria de N números de m bits (por ejemplo, $m=32$ para flotantes en simple precisión) como una matriz de $N \times m$, y reorganiza la data tomando columnas de bits completas. Tras esta permuta, aplica una compresión LZ (usualmente LZ4) sobre la secuencia resultante.

El fundamento es que, incluso si los valores originales parecen aleatorios, puede haber *correlaciones ocultas* dentro de ciertos planos de bits. Por ejemplo, en datos científicos muchos valores comparten el mismo exponente o presentan bits altos similares, lo que tras Bitshuffle

se manifiesta como secuencias repetitivas (*runs* de ceros o unos) más fáciles de comprimir mediante LZ4. A diferencia de métodos predictivos, Bitshuffle no asume continuidad entre elementos adyacentes; su efectividad radica en convertir cualquier correlación intra-byte (dentro de la representación binaria de cada valor) en redundancias horizontales a lo largo de la secuencia transformada. En datos de alta entropía pura (bits totalmente aleatorios), Bitshuffle no puede crear patrones donde no los hay; con todo, en vectores de estado cuántico puede explotar pequeñas irregularidades: si los qubits están normalizados, muchos amplitudes comparten signos o exponentes comunes, generando columnas de bits constantes que LZ4 comprime muy bien. En la práctica, Bitshuffle+LZ4 logra relaciones de compresión superiores a compresores genéricos en escenarios científicos sin reducción de precisión, a la vez que mantiene velocidades de descompresión del orden de gigabytes por segundo por núcleo.

4.4.4 Transformación de Datos Tipados (TDT)

Una innovación reciente en compresión de flotantes es la *Transformación de Datos Tipados* (TDT) (Jamalidinan y Cheshmi, 2025), concebida para cerrar la brecha entre compresores genéricos y especializados en punto flotante. TDT no es un compresor en sí, sino una etapa de *preprocesamiento* adaptable que reorganiza los bytes de los datos antes de aplicar un compresor convencional (p. ej., Zstd o LZ4). La idea clave es agrupar y separar los bytes según su *nivel de entropía* y patrones estadísticos, en lugar de tratarlos uniformemente. En una representación IEEE 754, distintos campos (signo, exponente, mantisa) pueden poseer distribuciones diferentes; por ejemplo, en muchas series científicas los bytes de exponente tienden a repetirse más que los bytes de mantisa. TDT analiza el conjunto de datos (en bloques) extrayendo características estadísticas de cada posición de byte (entropía media, desviación, frecuencias de valores) y aplica *clustering* para decidir cuáles bytes se comportan de forma similar. Luego *reordena* la matriz de datos colocando juntos los bytes de comportamientos afines y separando los de entropía distinta. Esto produce flujos de datos más homogéneos que, al ser alimentados a un compresor genérico, permiten una codificación más eficiente.

Importante: TDT no altera ningún bit de la información (es totalmente reversible), solo cambia su disposición para exponer patrones que antes estaban enmascarados. En el contexto de vectores de estado cuántico —que suelen ser casos límite de alta aleatoriedad— TDT buscaría, por ejemplo, si ciertos bytes de mantisa comparten distribuciones o si el byte de exponente tiene menor entropía debido a normalizaciones. Incluso si los datos son mayormente aleatorios, esta técnica puede detectar *no uniformidades* sutiles entre componentes y agruparlas para facilitar su compresión. Resulta particularmente atractiva como capa previa porque ofrece un equilibrio: en *datasets* de baja entropía conserva beneficios de modelos flotantes (predicciones locales), y en *datasets* de alta entropía se comporta como un reordenamiento que potencia a los compresores genéricos, habilitando ganancias modestas de compresión y mejoras de rendimiento en los *throughputs* de compresión y descompresión.

4.5 Otros Métodos de Optimización de Memoria

Además de la compresión sin pérdida, se han desarrollado enfoques alternativos para reducir el consumo de memoria en simuladores cuánticos, incluyendo técnicas basadas en redes tensoriales, poda de estados y optimización HPC.

Uno de los primeros enfoques en esta dirección fue propuesto por Markov y Shi (2008), quienes introdujeron el uso de redes tensoriales para simular circuitos cuánticos mediante contracción optimizada de tensores. En este método, el circuito se representa como un grafo tensorial, y su simulación se optimiza contrayendo secuencias de tensores en un orden eficiente. Esto permite representar ciertos circuitos con memoria subexponencial, aunque la efectividad del método depende de la estructura del entrelazamiento.

Posteriormente, Boixo y cols. (2017) propusieron un modelo gráfico para circuitos cuánticos de baja profundidad basado en la eliminación de variables en grafos probabilísticos. Su técnica permitió simular circuitos aleatorios cercanos al régimen de supremacía cuántica en hardware clásico, extendiendo el tamaño de circuitos simulables sin necesidad de almacenar el estado cuántico completo. Sin embargo, este método pierde eficacia conforme aumenta la

profundidad del circuito.

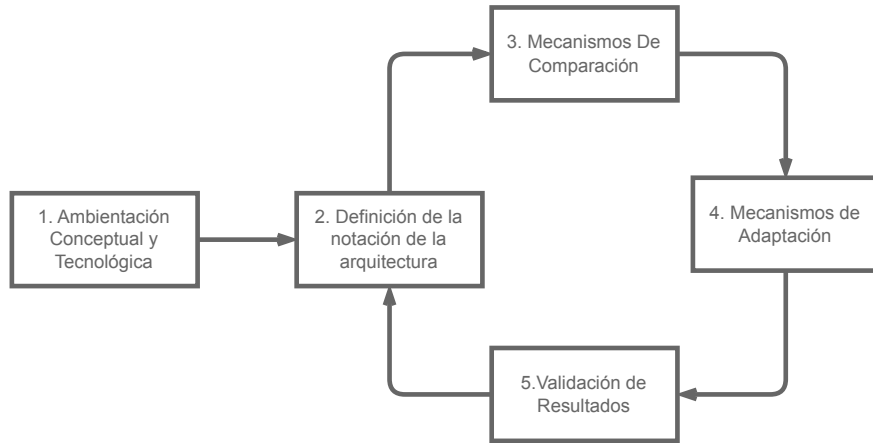
En paralelo, Pednault y cols. (2020) introdujeron técnicas de diferimiento de contracciones tensoriales en simulaciones HPC. Su propuesta permite manejar circuitos más grandes al intercambiar memoria por cómputo adicional, almacenando resultados intermedios en memoria secundaria y evitando la expansión total del estado cuántico en RAM. Esta estrategia ha sido clave en la simulación de circuitos con estructuras altamente no triviales, aunque su implementación requiere acceso a infraestructuras de supercomputación.

Finalmente, como se analizó previamente, la tesis doctoral de Díaz Toro (2025) introduce técnicas de compresión con pérdida de precisión en esquemas distribuidos de simulación cuántica. Si bien su propuesta constituye un hito en la reducción del consumo de memoria mediante bibliotecas como ZFP y SZ, el presente trabajo se orienta en una dirección distinta: se retoman fundamentos como el uso de vectores de estado y paralelismo distribuido con *MPI*, pero se propone como contribución original la exploración de técnicas de compresión **sin pérdida de precisión**. Esta línea de trabajo, no abordada en dicha tesis, busca preservar la fidelidad numérica total durante la simulación, evaluando el impacto empírico de algoritmos de compresión científica sin pérdida en el contexto de computación cuántica.

5 Metodología de la Investigación

Para este estudio, se ha seleccionado una metodología basada en investigación aplicada y evaluación experimental debido a la necesidad de diseñar, implementar y validar una estrategia de compresión sin pérdida de precisión en simuladores cuánticos. Dado que el problema a abordar implica la optimización del uso de memoria en simulaciones de alto consumo computacional, es fundamental seguir un enfoque sistemático que permita comparar la efectividad de la propuesta con otras estrategias existentes. Además, la metodología elegida permite realizar pruebas controladas sobre diferentes simuladores cuánticos y evaluar el impacto de la compresión en términos de memoria, rendimiento y fidelidad, garantizando así la reproducibilidad y validez de los resultados obtenidos.

Figura 2:
Metodología de investigación



5.1 Diseño

Comprende el estudio del estado del arte, la definición de requisitos y supuestos, la selección del simulador, y la planificación experimental y de integración técnica.

- **Revisión de literatura y estado del arte.** Levantamiento sistemático de técnicas de compresión sin pérdida de precisión para arreglos de punto flotante y su aplicabilidad en simulación cuántica de *vectores de estado*. Se identifican ventajas, limitaciones y brechas; se sintetizan los fundamentos teóricos y se establecen hipótesis de trabajo.
- **Criterios y procedimiento de selección del simulador cuántico.** Evaluación de alternativas (p. ej., QuEST, Intel-QS, qsim, Quantum++, TMFQS) con base en: (i) *apertura y modificabilidad* del código, (ii) *facilidad de integración* de estructuras/arrays personalizados para el vector de estado, (iii) *soporte a optimizaciones de memoria* y paralelismo (OpenMP/MPI/GPU), y (iv) *capacidad de instrumentación* (medición de memoria y tiempo). Se aplica una matriz de decisión ponderada y se documenta la trazabilidad de la elección.
- **Arquitectura de la solución y diseño de la estrategia de compresión.** Especificación de las estructuras de datos y puntos de inserción en el simulador (capa de

almacenamiento del vector de amplitudes y rutas críticas de acceso). Selección del/los algoritmo(s) de compresión sin pérdida de precisión o equivalentes numéricos que no alteren la fidelidad numérica de los estados cuánticos (p. ej., representaciones compactas o transformaciones reversibles), así como políticas de (des)compresión y su impacto esperado en memoria, tiempo y paralelismo.

5.2 Implementación y optimización del modelo

La estrategia de compresión seleccionada se implementará en el simulador cuántico elegido, modificando su sistema de almacenamiento y gestión de memoria. Se realizarán optimizaciones para garantizar un desempeño eficiente, evitando costos computacionales excesivos debido a la aplicación de la compresión. Durante esta etapa, se llevarán a cabo las siguientes actividades:

- Modificación del código fuente del simulador para integrar la compresión sin pérdida.
- Optimización del sistema de almacenamiento para reducir el uso de memoria sin afectar el rendimiento.
- Validación inicial de la implementación mediante pruebas con circuitos de prueba.

5.3 Evaluación experimental

Se diseñará un conjunto de experimentos para evaluar el impacto de la estrategia de compresión en diferentes circuitos cuánticos. Los experimentos permitirán determinar si se logra un balance adecuado entre eficiencia computacional y fidelidad. En esta fase se realizarán las siguientes actividades:

- Definición de un conjunto de circuitos de prueba para evaluar la estrategia de compresión:
 - Transformada de Fourier Cuántica (QFT)

- Algoritmo de búsqueda de Grover
- Ejecución de simulaciones con y sin compresión para medir su impacto en el rendimiento y el uso de memoria:
 - Tiempo de ejecución total (segundos)
 - Uso máximo de memoria (MB)
 - Fidelidad del estado final:

$$F(|\psi_r\rangle, |\psi_c\rangle) = |\langle\psi_r|\psi_c\rangle|^2.$$

- Análisis de los resultados obtenidos y comparación con enfoques tradicionales.

6 Simuladores cuánticos

6.1 Motivación y necesidad de un simulador base

El núcleo experimental de este trabajo consiste en diseñar e integrar técnicas de compresión *sin pérdida de precisión* sobre la estructura de datos más costosa en simulación cuántica tipo Schrödinger: el vector de estado (amplitudes complejas). Por lo tanto, el simulador seleccionado no es un detalle de implementación, sino una decisión metodológica que condiciona:

- la **viabilidad técnica** de insertar estructuras de almacenamiento comprimidas sin alterar la fidelidad numérica;
- el **costo de ingeniería** requerido para instrumentar mediciones reproducibles de memoria y tiempo;
- y la **capacidad de aislar variables** (compresión vs. paralelismo vs. optimizaciones internas) durante la evaluación experimental.

De hecho, dentro de la fase *Diseño* planteada en la metodología, se establece explícitamente que la selección debe basarse en: (i) apertura y modificabilidad del código, (ii) facilidad de integrar arreglos/estructuras personalizadas para el vector de estado, (iii) soporte a optimizaciones de memoria y paralelismo, y (iv) capacidad de instrumentación (medición de memoria y tiempo).

Bajo ese marco, se estudian cinco alternativas representativas por uso y disponibilidad en la comunidad: QuEST, Intel-QS, qsim, Quantum++ y TMFQSfullstate. Para asegurar trazabilidad, la decisión se formaliza con criterios y una matriz de decisión ponderada, de modo que la elección final sea justificable y reproducible.

6.2 Criterios de selección

Con el fin de elegir un simulador adecuado para integrar y evaluar compresión *sin pérdida de precisión* sobre el vector de estado, se definieron los criterios de la tabla 1. Estos criterios operacionalizan los lineamientos metodológicos del diseño (modificabilidad, integración de estructuras, optimización/paralelismo e instrumentación), y además incorporan requisitos prácticos de ingeniería para reducir riesgo técnico durante la implementación.

Tabla 1:

Criterios usados para la selección del simulador cuántico

	Criterio	Explicación	Peso
C1	Vector de estado apto para compresión sin pérdida	El simulador debe representar el vector de estado como arreglos contiguos de valores de punto flotante en doble precisión (amplitudes complejas), de manera que la compresión sin pérdida se pueda aplicar directamente sobre dicha estructura sin cambios semánticos.	0.10
C2	Facilidad de integración de estructuras/arrays personalizados	El código debe permitir reemplazar o envolver el almacenamiento del vector de amplitudes (p. ej. insertar un contenedor comprimido) con impacto controlado en el resto del simulador. Este criterio prioriza código modular y rutas críticas identificables.	0.25
C3	Soporte y afinidad con optimización de memoria	Se valora que el simulador esté concebido para experimentar con estrategias de memoria (p. ej. compresión, poda/pruning, particionamiento o variantes del almacenamiento), reduciendo esfuerzo para insertar y validar la estrategia propuesta.	0.20
C4	Complejidad de construcción y dependencias	Un proceso de compilación simple y pocas dependencias facilitan reproducibilidad y portabilidad de experimentos. Se penalizan toolchains complejas cuando incrementan el esfuerzo de integración e instrumentación.	0.10
C5	Capacidad de instrumentación (tiempo/memoria)	El simulador debe permitir medir de forma fina (por secciones) el consumo de memoria y el tiempo de ejecución, idealmente en puntos cercanos al acceso al vector de estado.	0.15
C6	Cobertura funcional mínima para benchmarks	Debe ser posible ejecutar circuitos estándar de evaluación (p. ej. QFT y/o Grover) con un conjunto mínimo de compuertas, para poder comparar desempeño con y sin compresión.	0.10
C7	Paralelismo relevante (CPU / OpenMP / MPI)	Se considera valioso poder evaluar la interacción compresión-paralelismo. Se prioriza soporte a OpenMP/MPI cuando el objetivo incluye escalamiento o comparación bajo concurrencia.	0.10

Los pesos fueron definidos para privilegiar la **integración del almacenamiento del vector de estado** (C2) y la **afinidad con experimentación en memoria** (C3), dado

que son los factores que más reducen el riesgo técnico de insertar compresión sin pérdida sin distorsionar el comportamiento del simulador.

6.3 Alternativas de simuladores evaluadas

A continuación, se resumen las herramientas evaluadas, enfatizando sus objetivos y características relevantes para esta tesis.

- **QuEST (Quantum Exact Simulation Toolkit):** Es un simulador de alto rendimiento para vectores de estado y matrices de densidad, con soporte híbrido de paralelismo que puede incluir multihilo, GPU y ejecución distribuida. Se caracteriza por estar orientado a rendimiento en múltiples arquitecturas de cómputo. (*The Quantum Exact Simulation Toolkit (QuEST) Documentation*, s.f.; *QuEST: Quantum Exact Simulation Toolkit*, s.f.)
- **Intel-QS (Intel Quantum Simulator):** Simulador de circuitos optimizado para aprovechar arquitecturas multinúcleo y multinodo; usa representación completa del estado, y está diseñado para escalar mediante paralelismo y distribución. (*Intel Quantum Simulator (Intel-QS) Documentation*, s.f.)
- **qsim (Google):** Librería de C++/Python para simulación *full state-vector*, ampliamente usada en integración con Cirq. En su documentación se describe explícitamente como simulador de vector de estado completo, donde el costo crece proporcionalmente al número de amplitudes 2^n . (*qsim: Fast C++ and Python library for state-vector simulation of quantum circuits*, s.f.; *qsim Cirq Interface Documentation*, s.f.)
- **Quantum++:** Librería moderna en C++ (enfocada en facilidad de uso y portabilidad) para simulación de procesos cuánticos; se distribuye como librería *header-only* y ofrece ejecución multihilo, siendo útil para prototipado. (Gheorghiu, 2018)
- **TMFQSfullstate:** Prototipo de simulador implementado en C++ con enfoque explícito en estudiar estrategias de consumo de memoria (representación full-state, poda de

estados, paralelismo compartido y distribuido, y compresión). En particular, el trabajo metodológico que acompaña este simulador reporta escenarios con OpenMP, MPI y la incorporación de compresión para evaluar huella de memoria. (G. J. Díaz, 2024; G. Díaz, 2025; Díaz Toro, 2024)

En conjunto, las alternativas cubren desde herramientas HPC maduras (QuEST, Intel-QS, qsim) hasta librerías generales (Quantum++) y un prototipo deliberadamente diseñado para experimentar con estrategias de memoria (TMFQSfullstate).

6.4 Matriz de evaluación ponderada

Se aplicó una matriz de decisión ponderada para evaluar cada alternativa según los criterios de la tabla 1. La calificación se asignó en una escala discreta de 1 a 5, donde 5 indica cumplimiento excelente del criterio y 1 un cumplimiento deficiente para los objetivos de integración e instrumentación de compresión sin pérdida.

Tabla 2:

Evaluación de alternativas (escala 1–5) y total ponderado

Criterio	Peso	TMFQS	QuEST	Intel-QS	qsim	Quantum++
C1	0.10	5	4	4	4	4
C2	0.25	5	3	2	2	3
C3	0.20	5	3	3	2	2
C4	0.10	4	3	2	2	5
C5	0.15	5	3	2	2	3
C6	0.10	3	5	5	5	4
C7	0.10	4	5	5	4	3
Total	1.00	4.60	3.50	3.00	2.70	3.10

Los resultados muestran que, aunque QuEST, Intel-QS y qsim ofrecen excelente cobertura funcional y paralelismo, la elección para esta tesis debe priorizar el costo de modificación del almacenamiento del vector de estado y la instrumentación cercana a rutas críticas. Bajo

ese objetivo, TMFQsfullstate obtiene el mayor puntaje global, principalmente por C2, C3 y C5.

6.5 Selección del simulador: TMFQsfullstate

Partiendo de la matriz de la tabla 2, se selecciona **TMFQsfullstate** como simulador base para esta tesis.

La decisión se justifica por cuatro razones principales:

- **Diseño orientado a experimentación de memoria (C3):** TMFQsfullstate se presenta como un prototipo cuyo foco explícito es evaluar consumo de memoria bajo distintas estrategias (full-state, OpenMP, MPI y escenarios con compresión), lo cual se alinea directamente con el objetivo de insertar compresión sin pérdida sobre el vector de estado. (G. J. Díaz, 2024; G. Díaz, 2025)
- **Alta modificabilidad del núcleo (C2) e instrumentación (C5):** al ser un prototipo compacto y de propósito investigativo, reduce el esfuerzo necesario para ubicar puntos de inserción (capa de almacenamiento del vector de amplitudes y rutas críticas) y para instrumentar mediciones finas de tiempo/memoria, tal como se exige en la fase de diseño metodológico.
- **Compatibilidad con arreglos de doble precisión (C1):** el material metodológico asociado al prototipo describe implementaciones basadas en arreglos en doble precisión para compuertas y estructuras numéricas, lo que facilita plantear compresión sin pérdida sobre datos de punto flotante sin introducir conversiones o cuantizaciones. (G. J. Díaz, 2024; G. Díaz, 2025)
- **Escenarios comparables de paralelismo (C7):** la disponibilidad de variantes con OpenMP/MPI permite evaluar la interacción entre compresión y paralelismo, y contrastar contra simuladores HPC sin que el paralelismo sea un factor ausente en el entorno experimental. (G. J. Díaz, 2024)

En consecuencia, TMFQSfullstate ofrece el mejor balance entre *control experimental* y *esfuerzo de integración*, minimizando riesgo técnico para implementar y evaluar compresión sin pérdida en el vector de estado, manteniendo al mismo tiempo una base suficientemente realista para comparar memoria y rendimiento.

7 Evaluación de algoritmos de compresión sin pérdida para TMFQSfullstate

7.1 Motivación y contexto de evaluación

En este proyecto, la compresión *sin pérdida de precisión* se plantea como un mecanismo para reducir la huella de memoria asociada a los datos numéricos del simulador, de modo que sea posible ampliar la escala de simulación y estudiar el costo-beneficio en términos de memoria, tiempo de ejecución y estabilidad numérica. La literatura ha mostrado que la compresión puede contribuir de forma significativa a disminuir requerimientos de memoria en simulación cuántica a gran escala, habilitando experimentos que de otra manera serían inviables (Wu y cols., 2019b; Department of Computer Science, University of Chicago, 2020).

Bajo esta motivación, la selección del algoritmo de compresión se aborda como una decisión de diseño: se requiere un compresor que logre un buen balance entre ratio de compresión, velocidad, y costo adicional en recursos, de manera que su incorporación permita evaluar la propuesta de esta tesis en escenarios realistas.

7.2 Criterios de selección

Para integrar compresión de datos en TMFQSfullstate (simulador de estado completo de circuitos cuánticos), se definen los siguientes criterios técnicos, ordenados por prioridad:

Tabla 3:

Criterios para la selección de compresores sin pérdida en TMFQSfullstate

Criterio		Explicación	Peso
C1	Ratio de compresión	Un mayor factor de compresión permite representar el vector de estado con menor memoria, habilitando la simulación de más qubits con los mismos recursos. En simulación de estado completo, la memoria suele ser el cuello de botella principal, por lo que maximizar la reducción de memoria sin pérdida es el objetivo dominante; este énfasis es consistente con la motivación de compresión en simulación cuántica a gran escala reportada por Wu y cols. (2019b).	0.40
C2	Velocidad de compresión y descompresión	La simulación puede requerir ciclos frecuentes de compresión/descompresión, por lo que el algoritmo debe minimizar latencia y maximizar throughput. Se priorizan compresores con throughput de centenas de MB/s (o más) y con descompresión aún más rápida para no retrasar la recuperación de datos (lz4 contributors, s.f.; facebook/zstd contributors, s.f.; Snappy project, s.f.; MaskRay, 2025).	0.20
C3	Bajo overhead de memoria	Dado el crecimiento exponencial del estado, el compresor no debe demandar estructuras internas (buffers/diccionarios/tablas) que incrementen sustancialmente el uso total de RAM. Se favorecen métodos que operen en streaming o por bloques pequeños (lz4 contributors, s.f.; Zstandard project, s.f.).	0.15
C4	Compatibilidad con procesamiento paralelo	En escenarios multi-core o distribuidos (p. ej., MPI en HPC), interesa que el compresor soporte multihilo de forma nativa o, como mínimo, que sea fácilmente paralelizable por bloques sin degradaciones significativas de ratio o throughput (facebook/zstd contributors, s.f.).	0.15
C5	Integración en C++ con TMFQSfullstate	El algoritmo debe estar disponible como librería utilizable desde C/C++ y ser suficientemente madura, con soporte y licencia permisiva, para minimizar riesgo de integración y mantenimiento (lz4 contributors, s.f.; facebook/zstd contributors, s.f.; google/snappy contributors, s.f.).	0.10

Los pesos se asignan para operacionalizar el orden de prioridad definido: se privilegia el ratio (C1) como objetivo dominante, seguido por el costo temporal (C2), y luego por restricciones de ingeniería relevantes para ejecución a gran escala e integración (C3–C5).

7.3 Alternativas de compresión sin pérdida consideradas

Se consideran compresores sin pérdida con disponibilidad en C/C++ y orientación a alto rendimiento, debido a su idoneidad para escenarios donde la compresión y descompresión pueden ejecutarse de manera recurrente.

- **LZ4:** Compresor enfocado en velocidad. El proyecto reporta compresión superior a 500 MB/s por núcleo y descompresión en múltiples GB/s por núcleo (frecuentemente limitada por la velocidad de la RAM) (lz4 contributors, s.f.). Su ratio suele ser moderado, a cambio de overhead reducido y una API simple, lo que facilita integración y paralelización por bloques (lz4 contributors, s.f.).
- **Zstandard (Zstd):** Compresor moderno que busca balancear velocidad con ratios elevados. Su objetivo explícito es ofrecer compresión comparable a zlib con mayor rendimiento, además de un rango amplio de niveles para ajustar el compromiso velocidad/ratio (facebook/zstd contributors, s.f.; Zstandard project, s.f.). Un rasgo relevante es el soporte nativo para compresión multihilo en la librería (facebook/zstd contributors, s.f.).
- **Snappy:** Biblioteca de compresión orientada a baja latencia. Se reportan velocidades típicas de ~ 250 MB/s en compresión y ~ 500 MB/s en descompresión en un solo núcleo (dependiente del hardware) (google/snappy contributors, s.f.; Snappy project, s.f.). Su ratio suele ser inferior a gzip/zlib, dado que privilegia throughput evitando etapas de entropía más costosas (Snappy project, s.f.).
- **zlib/Deflate (Gzip):** Solución estándar ampliamente disponible. Presenta ratios típicos que con frecuencia se sitúan entre 2:1 y 5:1 dependiendo del dato y del nivel de compresión (zlib authors, 2022). Sin embargo, su rendimiento suele ser menor que alternativas modernas orientadas a tiempo real (zlib authors, 2022).

Existen alternativas con ratios potencialmente superiores (p. ej. LZMA/XZ o Bzip2), pero

con mayor costo de CPU y latencia, por lo que son menos atractivas cuando el rendimiento es un requisito relevante en la ejecución del simulador (MaskRay, 2025).

7.4 Matriz de evaluación ponderada

La selección se formaliza mediante una matriz ponderada basada en los criterios de la Tabla 3. Se utiliza una escala discreta de 1 a 5, donde 5 representa el mejor desempeño relativo frente al criterio evaluado. Los puntajes se asignan con base en la documentación citada y la comparación cualitativa reportada en dichas fuentes.

Tabla 4:
Evaluación de alternativas de compresión (escala 1–5) y total ponderado

Criterio	Peso	Zstd	LZ4	Snappy	zlib
C1 (Ratio)	0.40	4	3	2	5
C2 (Velocidad)	0.20	4	5	4	2
C3 (Overhead memoria)	0.15	3	5	5	4
C4 (Paralelismo)	0.15	5	4	4	3
C5 (Integración C/C++)	0.10	5	5	4	5
Total	1.00	4.10	4.05	3.35	3.95

La tabla refleja el compromiso clásico entre ratio y velocidad: zlib tiende a maximizar la compresión, pero penaliza en throughput (zlib authors, 2022). LZ4 y Snappy destacan por velocidad y bajo overhead, usualmente con ratios inferiores a zlib y Zstd (lz4 contributors, s.f.; google/snappy contributors, s.f.; Snappy project, s.f.). Zstd se ubica como una alternativa de equilibrio, combinando ratios altos con rendimiento competitivo y soporte multihilo (facebook/zstd contributors, s.f.; Zstandard project, s.f.).

Como referencia complementaria, una comparación práctica de ratios en datos generales reporta el orden $\text{zlib} > \text{Zstandard} > \text{LZ4} > \text{Snappy}$ (StarRocks Documentation, s.f.), y benchmarks recientes ilustran el espectro de compromisos entre ratio, velocidad y memoria (MaskRay, 2025).

7.5 Selección del algoritmo recomendado

Con base en el total ponderado de la Tabla 4, se recomienda **Zstandard (Zstd)** como opción principal. El resultado se explica por su balance entre ratio y velocidad, y por funcionalidades prácticas para escenarios de cómputo intensivo, como el soporte de compresión multihilo en la librería (facebook/zstd contributors, s.f.; Zstandard project, s.f.).

Dado que el propósito de este trabajo es reducir el consumo de memoria sin afectar la exactitud de los resultados, la elección del compresor debe favorecer un ratio suficiente para que la reducción sea significativa, sin introducir una penalización temporal que anule el beneficio. En este sentido, zlib ofrece ratios competitivos, pero típicamente con menor rendimiento (zlib authors, 2022), mientras que LZ4 ofrece excelente velocidad con ratios usualmente menores (lz4 contributors, s.f.). Por ello, Zstd se adopta como el mejor compromiso para la evaluación experimental.

Finalmente, se reconoce que el desempeño real depende de la distribución de los datos a comprimir; existen casos donde un algoritmo supera a otro según los patrones presentes, como se discute en experiencias prácticas reportadas por usuarios (r/zfs, 2024). Por esta razón, además de adoptar Zstd como candidato principal, se mantiene LZ4 como referencia de baja latencia para comparación experimental.

8 Patrones en vectores de estado de algoritmos cuánticos relevantes para compresión

8.1 Introducción

La simulación cuántica de estado completo representa el sistema mediante un vector de amplitudes complejas cuyo tamaño crece como 2^n (siendo n el número de qubits). En este contexto, la *compresibilidad* del vector depende fuertemente de los patrones numéricos que generan los algoritmos: simetrías, valores repetidos, regiones con ceros exactos o distribuciones altamente estructuradas tienden a favorecer la compresión; por el contrario, estados

densos con amplitudes variadas y poco redundantes tienden a reducir la efectividad de compresores sin pérdida.

En las subsecciones siguientes se describen patrones típicos de varios algoritmos conocidos y su impacto esperado en compresión de vectores de estado.

8.2 Búsqueda de Grover: amplitudes uniformes y estados de baja entropía

Grover inicia preparando una superposición uniforme sobre $N = 2^n$ estados base, usualmente aplicando compuertas de Hadamard a cada qubit. En esa preparación ideal, todas las amplitudes tienen el mismo valor (magnitud constante), por lo que el vector de estado queda compuesto por un único valor repetido N veces, un caso altamente favorable para compresión sin pérdida (PennyLane, 2023).

Tras cada iteración (oráculo + difusión), la estructura permanece altamente regular: cuando existe un único elemento marcado, el vector puede describirse con dos categorías principales de amplitud (una asociada al estado marcado y otra común al resto de estados no marcados). Esta simetría reduce el número de valores distintos en el vector, lo cual incrementa la redundancia explotable por compresión.

De manera empírica, se ha reportado que la compresión permite escalar simulaciones de Grover significativamente más allá de lo que permitiría almacenar el vector sin comprimir. Por ejemplo, Wu y cols. muestran una reducción drástica del requerimiento de memoria para una simulación de Grover de 61 qubits al combinar técnicas de compresión con ejecución en supercomputación (Wu y cols., 2019b).

8.3 Transformada Cuántica de Fourier: patrones de fase y baja redundancia

La Transformada Cuántica de Fourier (QFT) aplicada a un estado base $|k\rangle$ produce una superposición con magnitudes uniformes pero fases dependientes del índice. En su forma

ideal, la amplitud de cada componente $|x\rangle$ puede expresarse como:

$$a_x = \frac{1}{\sqrt{N}} e^{\frac{2\pi i k x}{N}},$$

lo cual implica que, aunque la magnitud sea constante, las partes real e imaginaria varían de forma oscilatoria con x .

Desde la perspectiva de almacenamiento, este comportamiento tiende a generar arreglos *densos* donde la mayoría de entradas son no nulas y muchas de ellas difieren entre sí. En consecuencia, la redundancia directa (por repetición exacta) suele ser baja para compresión estrictamente sin pérdida. En un estudio de integración de compresión en simulación cuántica se observa que, a medida que avanza un circuito QFT, el ratio de compresión puede caer desde valores altos en etapas iniciales hasta valores cercanos a 1 (poca o nula compresión efectiva) en etapas finales (Wu, Di, y cols., 2018). Este comportamiento ilustra que los estados generados por QFT pueden ser un caso exigente para compresión sin pérdida, especialmente cuando el circuito densifica el vector de estado.

8.4 Algoritmo de Shor: periodicidad, dispersión y picos

Shor combina (i) una etapa de exponenciación modular que induce periodicidad y (ii) una QFT que transforma esa periodicidad en una distribución con picos relacionados con el período. En términos cualitativos, antes de aplicar QFT suelen aparecer patrones donde solo ciertos índices compatibles con una estructura periódica presentan amplitud significativa; dependiendo de la instancia, esto puede producir regiones con ceros exactos o repeticiones regulares que favorecen la compresión.

Tras QFT, la distribución tiende a concentrarse alrededor de múltiplos específicos asociados a la frecuencia/período, generando picos dominantes y muchos valores de magnitud pequeña. En compresión estrictamente sin pérdida, la ganancia depende de si existen ceros exactos o repeticiones; cuando predominan muchos valores pequeños pero distintos, la re-

dundancia disminuye. Estos escenarios se alinean con el fenómeno observado en circuitos que incrementan progresivamente la densidad del estado: conforme aparecen más amplitudes no nulas y variadas, el ratio de compresión disminuye (Wu, Di, y cols., 2018).

8.5 Deutsch–Jozsa y Simon: superposiciones estructuradas y esparsidad

Varios algoritmos tempranos construyen superposiciones altamente estructuradas mediante oráculos e interferencia.

En Deutsch–Jozsa, la computación induce cancelaciones que llevan el estado hacia un resultado fuertemente concentrado en un subconjunto pequeño de estados base (en casos ideales, un estado base dominante). Este tipo de concentración implica esparsidad (muchos ceros exactos), lo que es altamente favorable para compresión sin pérdida.

En Simon, tras medir el registro de salida, el registro de entrada colapsa a una superposición de dos estados relacionados por una máscara secreta. Ese estado intermedio es extremadamente esparso (dos entradas no nulas) y, por tanto, muy compresible. Posteriormente, la aplicación de Hadamards produce una superposición uniforme sobre un subconjunto que satisface una condición lineal; nuevamente aparecen patrones de ceros exactos y amplitudes repetidas. Una exposición divulgativa describe este comportamiento estructurado y la naturaleza de las superposiciones restringidas en Simon (Gupta, 2025).

8.6 Circuitos variacionales y circuitos aleatorios: entre regularidad y alta entropía

En aplicaciones NISQ, algunos circuitos variacionales producen distribuciones de amplitud sesgadas por la estructura del problema, lo cual puede inducir regularidades (por ejemplo, subconjuntos dominantes o restricciones que eliminan estados). Esto puede mejorar la compresibilidad frente a escenarios plenamente aleatorios.

Por el contrario, circuitos aleatorios profundos tienden a generar estados altamente entrelazados y con amplitudes densas, donde la diversidad de valores reduce la redundancia.

En simulación con compresión, este fenómeno se manifiesta como una caída del ratio a medida que el circuito genera más amplitudes no nulas y variadas, tal como se evidencia en evaluaciones de compresión sobre circuitos que densifican el vector de estado (por ejemplo, QFT) (Wu, Di, y cols., 2018).

8.7 Implicaciones para compresión sin pérdida en simulación de estado completo

Los ejemplos anteriores sugieren una heurística práctica: algoritmos que preservan simetrías fuertes (valores repetidos), esparsidad (ceros exactos) o distribuciones con pocas categorías de amplitud tienden a ser más favorables para compresión sin pérdida. En cambio, transformaciones que producen estados densos y con amplitudes altamente variadas limitan la efectividad de compresores sin pérdida.

Esta relación patrón-compresibilidad es un insumo útil para diseñar experimentos y seleccionar benchmarks representativos al evaluar técnicas de reducción de memoria en simuladores cuánticos. Resultados experimentales sobre compresión en simulación de circuitos muestran precisamente que, en presencia de estados densos y sin reglas simples de distribución, el ratio puede acercarse a 1 (Wu, Di, y cols., 2018), mientras que en algoritmos con alta regularidad (como Grover) pueden observarse reducciones de memoria sustanciales (Wu y cols., 2019b).

Referencias

- Arute, F., Arya, K., Babbush, R., Bacon, D., Bardin, J. C., Barends, R., ... Martinis, J. M. (2019). Quantum supremacy using a programmable superconducting processor. *Nature*, 574(7779), 505–510. Descargado de <https://www.nature.com/articles/s41586-019-1666-5> doi: 10.1038/s41586-019-1666-5
- Boixo, S., Isakov, S. V., Smelyanskiy, V. N., y Neven, H. (2017). Simulation of low-depth quantum circuits as complex undirected graphical models. *arXiv preprint arXiv:1712.05384*. Descargado de <https://arxiv.org/abs/1712.05384>
- Burtscher, M., y Ratanaworabhan, P. (2009). Fpc: A high-speed compressor for double-precision floating-point data. *IEEE Transactions on Computers*, 58(1), 18–31. Descargado de <https://userweb.cs.txstate.edu/~mb92/papers/tc09.pdf> doi: 10.1109/TC.2008.131
- Chen, J., Zhang, F., Huang, C., Newman, M., y Shi, Y. (2018). *Classical simulation of intermediate-size quantum circuits*. Descargado de <https://arxiv.org/abs/1805.01450>
- Díaz, G. (2025). Memory management strategies for software quantum state simulation. *MDPI*, 7(3), 41. (Describe estrategias de memoria y referencia al prototipo TMFQS)
- Díaz, G. J. (2024). How to build a software quantum simulator. *Preprints.org*. (Incluye escenarios full-state con OpenMP, MPI y compresión)
- Díaz Toro, G. J. (2024). *Tmfqsfullstate: Prototype quantum computing simulator (full state vector)*. GitHub repository.
- Díaz, G., Steffenel, L. A., Barrios, C. J., y Couturier, J.-F. (2024). *How to build a software quantum simulator*. Preprints. Descargado de <https://doi.org/10.20944/preprints202409.1497.v1> doi: 10.20944/preprints202409.1497.v1
- Díaz Toro, G. J. (2025). *Gestión de memoria en simuladores de computación cuántica de software* (Tesis doctoral, Universidad Industrial de Santander). Descargado 2025-05-13,

- de <https://noesis.uis.edu.co/items/357883df-3223-45b0-9848-f7681c5b0511>
- Gheorghiu, V. (2018). Quantum++: A modern c++ quantum computing library. *PLOS ONE*.
- Gupta, N. (2025, diciembre). *Simon's algorithm: Cracking the hidden code*. Medium (The Quantum Ladder). Descargado de <https://medium.com/the-quantum-ladder/simons-algorithm-cracking-the-hidden-code-f55c2d9c293e> (Accedido: 2026-01-26)
- Huffman, N., Pavlichin, D., y Weissman, T. (2024). *Lossy compression for schrödinger-style quantum simulations*. Descargado de <https://arxiv.org/abs/2401.11088>
- Intel quantum simulator (intel-qs) documentation*. (s.f.). ReadTheDocs.
- Jamalidinan, S., y Cheshmi, K. (2025). *Floating-point data transformation for lossless compression*. Descargado de <https://arxiv.org/abs/2506.18062> doi: 10.48550/arXiv.2506.18062
- Jaques, S., y Häner, T. (2021). *Leveraging state sparsity for more efficient quantum simulations*. Descargado de <https://arxiv.org/abs/2105.01533>
- Li, S., Kimura, Y., Sato, H., Yu, J., y Fujita, M. (2023). *Parallelizing quantum simulation with decision diagrams*. Descargado de <https://arxiv.org/abs/2312.01570>
- Lindstrom, P., y Isenbarg, M. (2006). Fast and efficient compression of floating-point data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5), 1245–1250. Descargado de <https://pubmed.ncbi.nlm.nih.gov/17080858/> doi: 10.1109/TVCG.2006.143
- Markov, I. L., Fatima, A., Isakov, S. V., y Boixo, S. (2018). *Quantum supremacy is both closer and farther than it appears*. Descargado de <https://arxiv.org/abs/1807.10749>
- Markov, I. L., y Shi, Y. (2008). Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3), 963–981. Descargado de <https://arxiv.org/abs/quant-ph/0511069> doi: 10.1137/050644756
- Masui, K., Amiri, M., Connor, L., Deng, M., Fandino, M., Hoefer, C., ... Vanderlinde, K.

- (2015). A compression scheme for radio data in high performance computing. *arXiv preprint arXiv:1503.00638*. Descargado de <https://arxiv.org/abs/1503.00638> (Describes the Bitshuffle filter and HDF5 plugin) doi: 10.48550/arXiv.1503.00638
- Miller, D. M., y Thornton, M. A. (2006). QMDD: A decision diagram structure for reversible and quantum circuits. En *Proceedings of the 36th international symposium on multiple-valued logic (ISMVL'06)* (p. 30). Descargado de <https://ieeexplore.ieee.org/document/1623982/> doi: 10.1109/ISMVL.2006.35
- Pednault, E., Gunnels, J. A., Nannicini, G., Horesh, L., Magerlein, T., Solomonik, E., ... Wisnieff, R. (2020). *Pareto-efficient quantum circuit simulation using tensor contraction deferral*. Descargado de <https://arxiv.org/abs/1710.05867>
- PennyLane. (2023). *Grover's algorithm / pennylane demos*. Web tutorial. Descargado de https://pennylane.ai/qml/demos/tutorial_grovers_algorithm (Accedido: 2026-01-26)
- qsim cirq interface documentation*. (s.f.). Quantum AI documentation.
- qsim: Fast c++ and python library for state-vector simulation of quantum circuits*. (s.f.). GitHub repository.
- The quantum exact simulation toolkit (quest) documentation*. (s.f.). Project documentation website.
- Quest: Quantum exact simulation toolkit*. (s.f.). GitHub repository.
- Song, Y., Li, Z., y Wu, J. (2023). *Mera: Memory reduction and acceleration for quantum circuit simulation via redundancy exploration*. Descargado de <https://arxiv.org/abs/2411.15332>
- Viamontes, G. F., Markov, I. L., y Hayes, J. P. (2005). Graph-based simulation of quantum computation in the density matrix representation. *Quantum Information & Computation*, 5(2), 113–130. Descargado de <https://arxiv.org/abs/quant-ph/0403114>
- Vidal, G. (2003). Efficient classical simulation of slightly entangled quantum computations. *Phys. Rev. Lett.*, 91(14), 147902. doi: 10.1103/PhysRevLett.91.147902

- Vinkhuijzen, L., Coopmans, T., Elkouss, D., Dunjko, V., y Laarman, A. (2023, septiembre). LIMDD: A Decision Diagram for Simulation of Quantum Computing Including Stabilizer States. *Quantum*, 7, 1108. Descargado de <https://quantum-journal.org/papers/q-2023-09-11-1108/> doi: 10.22331/q-2023-09-11-1108
- Wu, X.-C., Di, S., Cappello, F., Finkel, H., Alexeev, Y., y Chong, F. T. (2018). Amplitude-aware lossy compression for quantum circuit simulation. En *Proceedings of the 4th international workshop on data reduction for big scientific data (drbsd-4) at sc18*. IEEE.
- Wu, X.-C., Di, S., Dasgupta, E. M., Cappello, F., Finkel, H., Alexeev, Y., y Chong, F. T. (2019a). Full-state quantum circuit simulation by using data compression. En *Proceedings of the international conference for high performance computing, networking, storage and analysis (sc'19)*. IEEE/ACM. doi: 10.1145/3295500.3356155
- Wu, X.-C., Di, S., Dasgupta, E. M., Cappello, F., Finkel, H., Alexeev, Y., y Chong, F. T. (2019b). Full-state quantum circuit simulation by using data compression. *arXiv*. Descargado de <https://arxiv.org/abs/1911.04034> (Accedido: 2026-01-26)
- Wu, X.-C., Di, S., y cols. (2018). Amplitude-aware lossy compression for quantum circuit simulation. En *Sc18 workshops (drbsd)*. Descargado de https://sc18.supercomputing.org/proceedings/workshops/workshop_files/ws_drbsd112s1-file1.pdf (Accedido: 2026-01-26)
- Zhang, B., Fang, B., Ye, F., Gu, Y., Tallent, N., Tan, G., y Tao, D. (2024a). *Overcoming memory constraints in quantum circuit simulation with a high-fidelity compression framework*. Descargado de <https://arxiv.org/abs/2410.14088>
- Zhang, B., Fang, B., Ye, F., Gu, Y., Tallent, N., Tan, G., y Tao, D. (2024b). *Overcoming memory constraints in quantum circuit simulation with a high-fidelity compression framework*. Descargado de <https://arxiv.org/abs/2410.14088>